

山东大学
SHANDONG UNIVERSITY

密码分析学第四次实验报告

姓名：邱泽辉、江李阳、王健一

学号：202400460046、202400460042、202400460037

专业：密码科学与技术

日期：November 3, 2026

Contents

1 组员分工 **2**

2 实验环境 **2**

3 实验原理 **2**
- 3.1 DDT 与 LAT . . . 2
  - 3.1.1 差分分布表 . . . 2
  - 3.1.2 线性分布表 . . . 3
- 3.2 混合整数线性规划 . . . 3
- 3.3 差分与线性传播点集 . . . 4
- 3.4 二进制编码与模型验证 . . . 5
- 3.5 one-hot 法 . . . 5
- 3.6 逐点排除法 . . . 6
- 3.7 逻辑化简法 . . . 7
- 3.8 凸包法 . . . 8

4 实验内容 **10**
- 4.1 任务一：DDT 与 LAT 的建模及求解 . . . 10
  - 4.1.1 one-hot 法 . . . 12
  - 4.1.2 逐点排除法 . . . 15
  - 4.1.3 逻辑化简法 . . . 16
  - 4.1.4 凸包法 . . . 18
  - 4.1.5 四种方法的结果对比 . . . 20
- 4.2 任务二：Cipherfour 算法六轮最优差分与线性路线搜索 . . . 20
  - 4.2.1 实验步骤 . . . 20
  - 4.2.2 实验结果及分析 . . . 24

5 实验总结 **29**

1 组员分工

- 邱泽辉：完成任务一, 二代码检查; 完成实验报告最终检查和修改;
- 江李阳：完成任务一和实验报告对应章节撰写;
- 王健一：完成任务二和实验报告对应章节撰写.

2 实验环境

- 硬件环境：13th Gen Intel(R) Core(TM) i5-13420H 处理器，12 个逻辑处理器，主频约 2.1 GHz；内存 16 GB。
- 软件环境：Windows、MacOS、WSL Arch Linux，Visual Studio Code 集成开发环境。
- 编程语言：Python 3.14。
- 求解与建模工具:Gurobi 及其 Python 接口 `gurobipy` 13.0.3、PyEDA 0.29.0（Espresso 逻辑化简）、SageMath 10.9（凸包构造）。
- Python 依赖管理：`uv`,`pip`.

需要先安装 Gurobi, 免费版可以使用 2000 个变量和 2000 个约束条件, 足够满足本次实验.

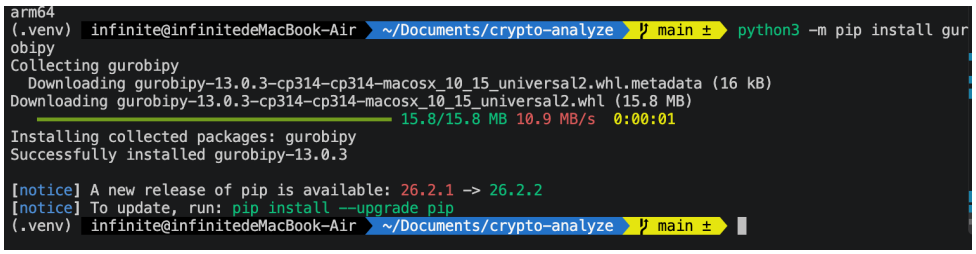


Figure 1: Gurobi 安装界面

3 实验原理

3.1 DDT 与 LAT

差分分布表（Differential Distribution Table，DDT）和线性分布表（Linear Approximation Table，LAT）分别记录 S 盒的差分传播关系与线性掩码关系。对于一个输入 m 比特、输出 n 比特的 S 盒 $S : \mathbb{F}_2^m \to \mathbb{F}_2^n$，两张表的输入、输出索引分别取遍相应向量空间。

3.1.1 差分分布表

给定输入差分 $\alpha \in \mathbb{F}_2^m$ 和输出差分 $\beta \in \mathbb{F}_2^n$，定义

$$IN_S(\alpha, \beta) = \{x \in \mathbb{F}_2^m \mid S(x) \oplus S(x \oplus \alpha) = \beta\}.$$

DDT 表项是满足该传播关系的输入个数：

$$\mathrm{DDT}[\alpha][\beta] = \#IN_S(\alpha, \beta).$$

因此，$\mathrm{DDT}[\alpha][\beta] > 0$ 表示该差分传播可达，表项的数值则记录其出现次数。

3.1.2 线性分布表

对于输入掩码 $\alpha \in \mathbb{F}_2^m$ 和输出掩码 $\beta \in \mathbb{F}_2^n$，定义

$$\mathrm{LAT}[\alpha][\beta] = \#\{x \in \mathbb{F}_2^m \mid \alpha \cdot x \oplus \beta \cdot S(x) = 0\} - 2^{m-1},$$

其中 $\cdot$ 表示模 2 内积。该表项等于满足线性关系的输入数与均匀随机基准值 2^{m-1} 之差。相应的线性偏差为

$$\epsilon(\alpha, \beta) = \Pr[\alpha \cdot x = \beta \cdot S(x)] - \frac{1}{2} = \frac{\mathrm{LAT}[\alpha][\beta]}{2^m}.$$

偏差的符号表示线性关系成立频率高于或低于 $1/2$；判断传播是否可达时，应保留所有非零表项，包括负值。

3.2 混合整数线性规划

混合整数线性规划（Mixed Integer Linear Programming，MILP）通过线性目标函数、线性约束以及整数变量描述离散优化问题。其一般形式可写为

$$\begin{aligned} \min \quad & c^{\mathsf{T}} u + d^{\mathsf{T}} v \\ \text{s.t.} \quad & Au + Bv \le b, \\ & u \in \mathbb{R}^n, \quad v \in \mathbb{Z}^m. \end{aligned}$$

其中，u 为连续变量，v 为整数变量；目标函数也可以设为最大化，约束中可以包含线性等式。若所有变量均为整数变量，则为整数线性规划；若变量进一步限制在 $\{0, 1\}$，则为 0–1 整数线性规划。本实验针对 S 盒传播建立的模型使用二进制变量。

0–1 整数线性规划的可行性判定在一般情形下是 NP 完全问题，相关优化问题具有 NP 困难性。这描述的是最坏情况下的复杂度, 但并不意味着每个实例都难以求解。图 2给出课程材料中的整数规划示例；对于本实验规模的传播模型，可由现成的整数规划求解器进行枚举和验证。

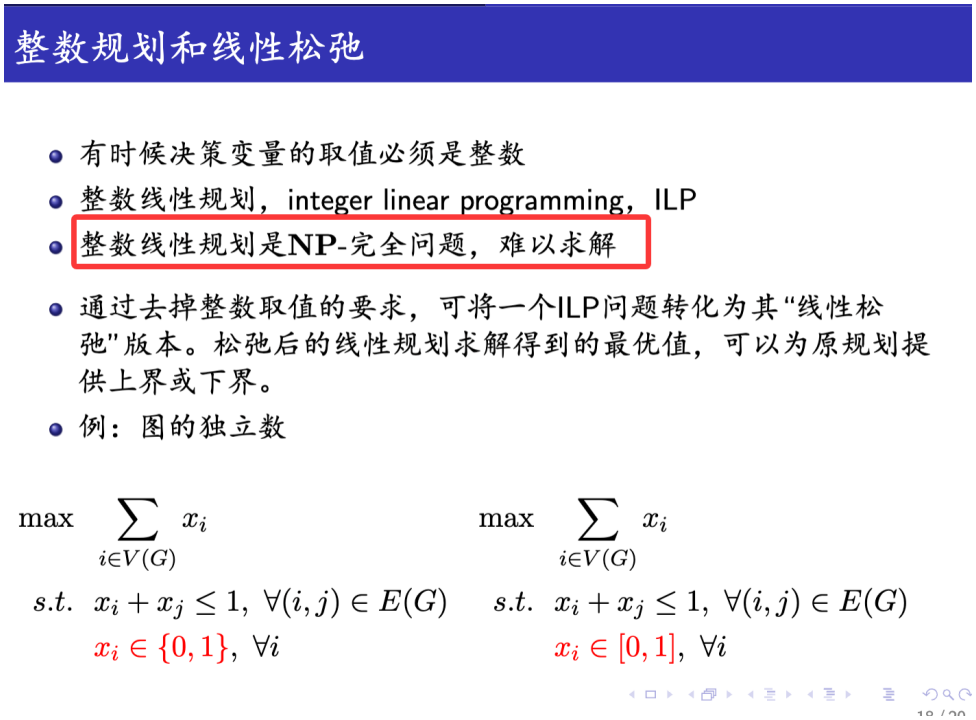


Figure 2: 整数线性规划示例

3.3 差分与线性传播点集

设本实验使用的 S 盒为 $S : \mathbb{F}_2^4 \to \mathbb{F}_2^4$。对于输入差分 a 和输出差分 b，DDT 表项 $\mathrm{DDT}[a][b]$ 统计满足 $S(x) \oplus S(x \oplus a) = b$ 的输入个数。相应的转移概率为

$$M[b,a] = \frac{\mathrm{DDT}[a][b]}{16}.$$

当 $\mathrm{DDT}[a][b] > 0$ 时，该差分转移可达；表项为 0 时，该转移不可达。对于输入掩码 a 和输出掩码 b，本实验采用

$$\mathrm{LAT}[a][b] = \#\{x \in \mathbb{F}_2^4 \mid a \cdot x \oplus b \cdot S(x) = 0\} - 8$$

定义 LAT。相应的相关度为

$$C[b,a] = \frac{\mathrm{LAT}[a][b]}{8}.$$

LAT 表项可以为正或负；只要 $\mathrm{LAT}[a][b] \neq 0$，相应的线性掩码传播就属于可达传播。因此，DDT 和 LAT 对应的传播点集分别定义为

$$V_{\mathrm{D}} = \{(a,b) \in \{0,\ldots,15\}^2 \mid \mathrm{DDT}[a][b] > 0\},$$
$$V_{\mathrm{L}} = \{(a,b) \in \{0,\ldots,15\}^2 \mid \mathrm{LAT}[a][b] \neq 0\}.$$

这里的点按“输入、输出”顺序记录。直接判断一个点是否属于有限点集不是线性约束，因此需要将该点集改写为线性整数模型，使模型在二进制变量上的可行解恰好对应这些传播点。

3.4 二进制编码与模型验证

对输入输出对 (a,b) 按高位在前的顺序展开，得到八维二进制向量

$$\mathrm{bits}(a,b) = (a_3,a_2,a_1,a_0,b_3,b_2,b_1,b_0) \in \{0,1\}^8.$$

四种方法均使用这一位序，并分别对 V_{D} 和 V_{L} 建立模型。对于只含原始变量的模型，其二进制可行点应当与相应点集的编码完全相同；若模型引入辅助变量，则比较其可行解在八个原始变量上的投影。

验证时将目标函数设为常数 0，并枚举模型的全部可行解。记枚举结果在八个传播变量上的投影为 S_{enum}，则

$$V \setminus S_{\mathrm{enum}}$$

表示遗漏的传播点，而

$$S_{\mathrm{enum}} \setminus V$$

表示多出的不可达点。只有在枚举已经完成且这两个差集均为空时，才能认为模型的二进制解集与目标点集相同.

3.5 one-hot 法

基本原理 one-hot 法为传播点集中的每个合法点设置一个二进制选择变量。变量取 1 表示当前解选择该点，取 0 表示当前解不选择该点。要求全部选择变量之和为 1，即可保证每个可行解恰好对应一个传播点。

数学建模 设合法点集为 V，每个点 $(a,b) \in V$ 按公共位序编码为 $\mathrm{bits}(a,b) \in \{0,1\}^8$。对每个点引入选择变量 $\lambda_{(a,b)}$，并以 $x \in \{0,1\}^8$ 表示八个传播坐标，则模型为

$$\begin{aligned}
\min \quad & 0 \\
\text{s.t.} \quad & \sum_{(a,b)\in V} \lambda_{(a,b)} = 1, \\
& x = \sum_{(a,b)\in V} \lambda_{(a,b)}\,\mathrm{bits}(a,b), \\
& \lambda_{(a,b)} \in \{0,1\}, \qquad\qquad \forall (a,b) \in V, \\
& x \in \{0,1\}^8.
\end{aligned} \tag{1}$$

第一条约束保证恰好选择一个点；第二条约束将该点的八个坐标赋给原始传播变量。由于每个坐标都是已知的 0 或 1 常数，坐标恢复关系为线性等式。

正确性分析 对模型的任意可行解，由于 $\lambda_{(a,b)}$ 均为二进制变量且其和为 1，存在唯一的 $(a^*,b^*) \in V$ 使得 $\lambda_{(a^*,b^*)} = 1$，其余选择变量均为 0。代入坐标等式可得

$$x = \mathrm{bits}(a^*,b^*),$$

因此模型不会产生 V 之外的点。

反过来，对任意 $(a,b) \in V$，令 $x = \mathrm{bits}(a,b)$、$\lambda_{(a,b)} = 1$，并令其余选择变量为 0，即可满足全部约束。因此，每个合法点都对应一个模型可行解，模型在 x 上的投影恰好为 V。

用于差分与线性传播 差分模型取 $V = V_{\mathrm{D}}$，选择变量分别对应 DDT 的非零位置；线性模型取 $V = V_{\mathrm{L}}$，选择变量分别对应 LAT 的非零位置。两种模型使用相同的坐标恢复约束，但各自建立独立的选择变量。全零点 $(0,0)$ 的选择变量也应保留，以表示未激活的 S 盒传播；表项的具体计数或偏差不作为本模型的优化目标。

3.6 逐点排除法

基本原理 逐点排除法直接描述合法点集的补集。设传播变量为 $x \in \{0,1\}^8$，合法点集为 V，则非法点集为

$$Q = \{0,1\}^8 \setminus V.$$

对于每个 $q = (q_1, \ldots, q_8) \in Q$，构造一条约束排除该点，同时保留与 q 不同的二进制点。

两个二进制向量的汉明距离等于其不同坐标的数量。由于 $x_j, q_j \in \{0,1\}$，其汉明距离可写为

$$d_H(x,q) = \sum_{j=1}^{8} |x_j - q_j| = \sum_{j:q_j=0} x_j + \sum_{j:q_j=1} (1 - x_j).$$

当且仅当 $x = q$ 时，$d_H(x,q) = 0$；若 $x \neq q$，则至少有一个坐标不同，因而 $d_H(x,q) \geq 1$。

数学建模 对每个非法点 $q \in Q$ 要求其汉明距离至少为 1，得到

$$\begin{aligned} \min \quad & 0 \\ \text{s.t.} \quad & \sum_{j:q_j=0} x_j + \sum_{j:q_j=1} (1 - x_j) \geq 1, \quad q \in Q, \\ & x_j \in \{0,1\}, \qquad\qquad\qquad\quad j = 1, \ldots, 8. \end{aligned}$$

将常数项移至右侧，也可写为

$$\sum_{j:q_j=0} x_j - \sum_{j:q_j=1} x_j \geq 1 - \sum_{j=1}^{8} q_j, \qquad q \in Q.$$

例如，若 $q = 0000\,0001$，排除该点的约束为

$$x_1 + x_2 + x_3 + x_4 + x_5 + x_6 + x_7 + (1 - x_8) \geq 1.$$

正确性分析 当 $x = q$ 时，约束左侧的汉明距离为 0，因此该非法点不满足相应约束而被排除。当 $x \neq q$ 时，至少有一个坐标不同，汉明距离至少为 1，所以该约束不会排除 q 以外的点。

由于 Q 包含所有非法二进制点，对每个 $q \in Q$ 分别加入上述约束后，全部非法点均被排除；而任何合法点 $x \in V$ 都不同于每个 $q \in Q$，因此满足所有约束。故模型的二进制可行解集合恰好为 V：

$$\left\{ x \in \{0,1\}^8 \;\middle|\; \sum_{j:q_j=0} x_j + \sum_{j:q_j=1} (1 - x_j) \geq 1,\ \forall q \in Q \right\} = V.$$

用于差分与线性传播 差分模型将 DDT 中取值为 0 的位置编码为 Q；线性模型将 LAT 中取值为 0 的位置编码为 Q。LAT 的正、负非零值均属于合法传播，因此构造补集时仅排除零值位置。该方法只使用八个原始二进制变量，每个非法点对应一条排除约束。

3.7 逻辑化简法

基本原理 逐点排除法为每个非法点添加一条约束。逻辑化简法先将非法点表示为布尔函数的取值为 1 的输入，再将函数化简为若干逻辑立方体（cube）。每个立方体可以覆盖多个非法点，因此可将逐点约束合并为较少的立方体约束。图 3展示课程材料中的立方体合并示意。

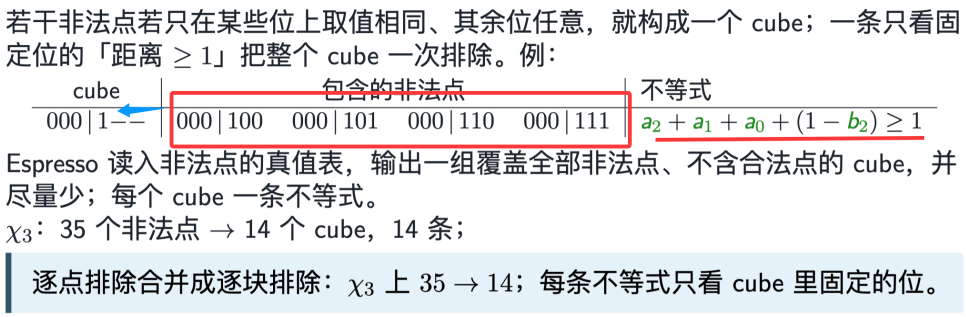


Figure 3: 非法点合并为逻辑立方体的示意图

非法点的布尔表示 设合法点集为 V，非法点集为 $Q = \{0,1\}^8 \setminus V$。定义八输入布尔函数

$$f_{\text{bad}}(x) = \begin{cases} 1, & x \in Q, \\ 0, & x \in V. \end{cases}$$

将该函数化简后，可得到由固定文字和通配文字组成的乘积项集合。用

$$c = (c_1, \ldots, c_8) \in \{0, 1, -\}^8$$

表示一个立方体，其中 $c_j = 0$ 或 1 表示该坐标固定为相应取值，$c_j = -$ 表示该坐标不受限制。定义

$$I_0(c) = \{j \mid c_j = 0\}, \qquad I_1(c) = \{j \mid c_j = 1\},$$

则立方体覆盖的点集为

$$B(c) = \left\{ x \in \{0,1\}^8 \;\middle|\; x_j = 0 \ (j \in I_0(c)),\ x_j = 1 \ (j \in I_1(c)) \right\}.$$

若化简得到的立方体集合为 $\mathcal{C}$，正确的覆盖关系为

$$\bigcup_{c \in \mathcal{C}} B(c) = Q. \tag{2}$$

该等式要求立方体并集覆盖全部非法点且不覆盖合法点；不同立方体之间允许重叠。

课程材料还给出了八输入、单输出 PLA 的表示方式：输入按 $(a_3, a_2, a_1, a_0, b_3, b_2, b_1, b_0)$ 排列，非法点对应输出 1，合法点对应输出 0，连字符表示通配位。本实验代码通过 PyEDA 的 `espresso_exprs` 接口化简布尔表达式，而不是读取外部 PLA 文件；代码显式构造非法点的最小项并取逻辑或，因此未列出的合法点取值为 0。化简结果中的每个乘积项即对应一个逻辑立方体。

立方体对应的线性约束 为排除一个立方体，只需要求当前点至少有一个固定坐标与该立方体不同。固定为 0 的坐标用 x_j 表示差异，固定为 1 的坐标用 $1-x_j$ 表示差异，通配坐标不进入约束。因此模型可写为

$$\begin{aligned} \min \quad & 0 \\ \text{s.t.} \quad & \sum_{j\in I_0(c)} x_j + \sum_{j\in I_1(c)} (1-x_j) \geq 1, \quad c \in \mathcal{C}, \\ & x_j \in \{0,1\}, \qquad\qquad\qquad\qquad j = 1,\ldots,8. \end{aligned} \tag{3}$$

移项后，每条约束也可写成

$$\sum_{j\in I_0(c)} x_j - \sum_{j\in I_1(c)} x_j \geq 1 - |I_1(c)|.$$

例如，立方体 $c = 0000\,000-$ 覆盖两个点 0000 0000 和 0000 0001，对应的排除约束为

$$x_1 + x_2 + x_3 + x_4 + x_5 + x_6 + x_7 \geq 1.$$

正确性分析 若 $x \in B(c)$，则 x 与 c 的全部固定坐标一致，式 (3)左侧为 0，因而违反该立方体对应的约束。若 $x \notin B(c)$，则至少有一个固定坐标不同，左侧至少为 1，故该约束不会排除立方体之外的点。

由覆盖关系 (2)可知，全部非法点至少属于一个立方体，因此都会违反相应约束；合法点不属于任何立方体，因而满足全部约束。模型的二进制可行解集合恰好为 V。该方法分别对 DDT 和 LAT 的零值位置化简；LAT 的非零正值与负值均保留为合法传播点。

3.8 凸包法

基本原理 集合 P 称为凸集，当且仅当对其中任意两点 $x, y \in P$ 及任意 $\lambda \in [0,1]$，线段上的点 $\lambda x + (1-\lambda)y$ 仍属于 P。有限点集 V 的凸包 $\mathrm{conv}(V)$ 是包含 V 的最小凸集，也可表示为 V 中点的所有凸组合：

$$\mathrm{conv}(V) = \left\{ \sum_{i=1}^{N} \theta_i v^{(i)} \;\middle|\; \theta_i \geq 0,\ \sum_{i=1}^{N} \theta_i = 1 \right\}.$$

在二维空间中，凸包的边界由线段构成；在高维空间中，其边界面由支撑超平面描述。图 4给出二维凸包示意。

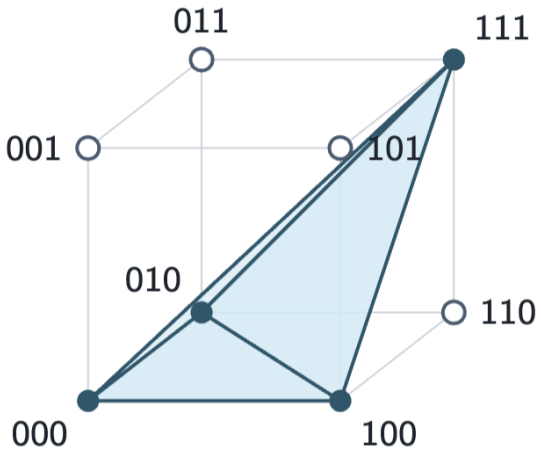


Figure 4: 有限点集的二维凸包示意图

八维多面体可用有限个线性不等式和等式表示，称为 H 表示。若 $P = \text{conv}(V)$，其 H 表示可写为

$$P = \{x \in \mathbb{R}^8 \mid Ax + b \geq 0,\ Cx + d = 0\}.$$

由此建立传播模型时，再限制 x 为二进制向量：

$$\begin{aligned} \min \quad & 0 \\ \text{s.t.} \quad & Ax + b \geq 0, \\ & Cx + d = 0, \\ & x \in \{0,1\}^8. \end{aligned} \tag{4}$$

二进制可行点的正确性 任意可达点 $v \in V$ 均属于 $\text{conv}(V)$，因此不会被完整 H 表示排除。还需说明凸包中的连续点不会成为新的二进制解。设 $x \in \text{conv}(V) \cap \{0,1\}^8$，则存在非负系数 θ_i 满足

$$x = \sum_{i=1}^{N} \theta_i v^{(i)}, \qquad \theta_i \geq 0, \qquad \sum_{i=1}^{N} \theta_i = 1.$$

若 $x_j = 0$，由于各 $v_j^{(i)} \geq 0$，所有满足 $\theta_i > 0$ 的点均须有 $v_j^{(i)} = 0$；若 $x_j = 1$，由于各 $v_j^{(i)} \leq 1$ 且权重之和为 1，所有满足 $\theta_i > 0$ 的点均须有 $v_j^{(i)} = 1$。因此，任意一个 $\theta_i > 0$ 的点都与 x 的八个坐标相同，即 $x = v^{(i)} \in V$。于是

$$\text{conv}(V) \cap \{0,1\}^8 = V.$$

该性质依赖于传播点采用二进制坐标编码；仅限制为一般整数坐标时，不能直接使用同一结论。

基于非法点覆盖的贪心筛选 完整 H 表示可能包含较多不等式。本实验对保留的八维二进制点集进行贪心筛选：所有等式均予以保留，并先从非法点集中移除不满足这些等

式的点，因为它们已被等式排除。设剩余非法点集为 Q_{eq}，完整 H 表示中的第 i 条不等式为

$$h_i(x) = A_i x + b_i \geq 0,$$

该不等式能够排除的点集为

$$R_i = \{q \in Q_{\text{eq}} \mid h_i(q) < 0\}.$$

每轮从尚未选用的不等式中，选择能排除当前最多剩余非法点的一条：

$$i^* = \arg\max_{i \in \text{pool}} |R_i \cap Q_{\text{remaining}}|.$$

将其加入保留集合后，更新

$$Q_{\text{remaining}} \leftarrow Q_{\text{remaining}} \setminus R_{i^*},$$

并重复上述过程，直至 $Q_{\text{remaining}} = \varnothing$。候选不等式按系数顺序确定并列时的选择，从而使筛选过程可重复。该贪心策略能在本次筛选中覆盖全部非法二进制点，但不保证得到约束数全局最少的子集。

约简模型的正确性 设贪心筛选保留的不等式构成矩阵 A' 和常数向量 b'，约简后的模型为

$$\begin{aligned} \min \quad & 0 \\ \text{s.t.} \quad & A'x + b' \geq 0, \\ & Cx + d = 0, \\ & x \in \{0,1\}^8. \end{aligned} \tag{5}$$

所有保留的不等式均来自完整 H 表示，因此每个可达点仍满足约简模型。对于任意非法二进制点 q，若其违反某条等式，则已被等式排除；若其满足全部等式，则属于初始的 Q_{eq}。贪心算法只有在全部剩余非法点均被覆盖后才终止，因此至少有一条保留不等式在 q 处取负值，q 也被约简模型排除。故约简模型在二进制变量上的可行点集仍恰好为 V。

差分模型与线性模型分别根据 DDT 和 LAT 的非零位置构造 V，独立生成完整凸包并筛选约束。筛选后的连续可行域可能大于完整凸包，但其二进制可行点集保持不变。

4 实验内容

4.1 任务一：DDT 与 LAT 的建模及求解

DDT 与 LAT 的直接计算 四种建模方法共用 `ddt_lat.py` 中的表格计算函数。计算 DDT 时，程序遍历每个输入差分 a 及每个输入 x，求出 $S(x) \oplus S(x \oplus a)$，并在对应的输出差分列累计出现次数：

```
for a in range(SIZE):
    for x in range(SIZE):
        b = SBOX[x] ^ SBOX[x ^ a]
        table[a][b] += 1
```

计算 LAT 时，程序遍历输入、输出掩码 a, b，统计满足 $a \cdot x = b \cdot S(x)$ 的输入数，再减去 $2^{4-1} = 8$。按位与结果的汉明重量奇偶性用于计算模 2 内积，核心代码如下：

```
equal_count = 0
for x in range(SIZE):
    input_parity = (a & x).bit_count() & 1
    output_parity = (b & SBOX[x]).bit_count() & 1
    if input_parity == output_parity:
        equal_count += 1
table[a][b] = equal_count - SIZE // 2
```

图 5给出了直接遍历计算得到的 DDT 和 LAT。表格的行、列分别对应输入和输出，索引均以十六进制表示。DDT 有 72 个非零表项，LAT 有 122 个非零表项；因此，差分模型和线性模型分别应保留 72 个与 122 个传播点。LAT 的负值同样表示非零偏差，构造合法点集时应保留所有非零表项；两张表均保留 $(0, 0)$。

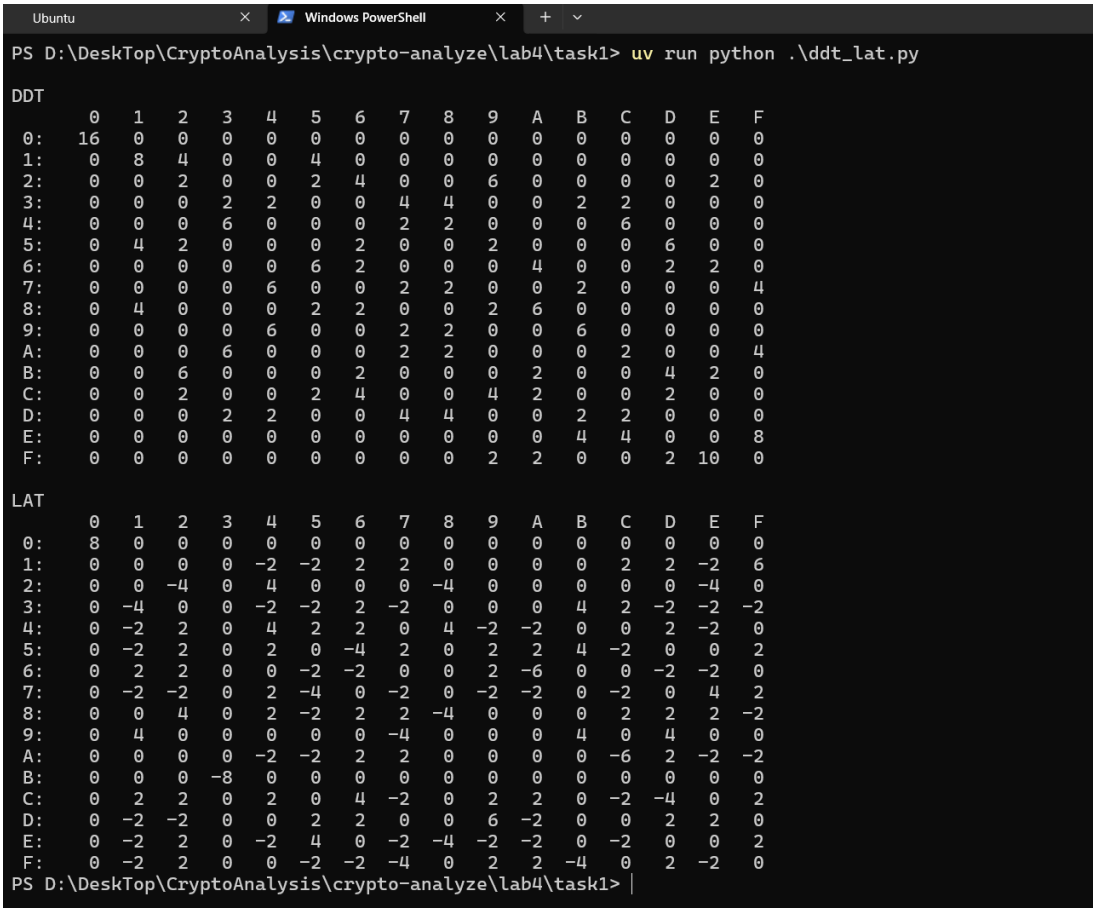


Figure 5: 直接遍历计算得到的 S 盒 DDT 与 LAT

统一编码与全解枚举 为使四种方法的求解结果可以直接比较，每个输入输出对均按大端序编码为

$$x = (a_3, a_2, a_1, a_0, b_3, b_2, b_1, b_0) \in \{0, 1\}^8.$$

其中输入和输出的高位均排列在前。各模型使用相同的位序，并将目标函数设为常数 0，以枚举所有满足约束的二进制解。解池的关键设置如下：

```
model.setObjective(0, gp.GRB.MINIMIZE)
model.Params.PoolSearchMode = 2
capacity = 2**8 + 1
model.Params.PoolSolutions = capacity
model.Params.MIPGap = 0
model.Params.MIPGapAbs = 0
model.optimize()
```

八位二进制向量共有 $2^8 = 256$ 种赋值，因而将解池容量设为 257，确保其严格大于完整赋值数。程序只有在求解状态为最优且解池未满时才确认枚举完成；随后逐个读取解，按照公共位序还原输入、输出，并以十六进制排序。每个解表示 DDT 或 LAT 中的一个非零位置，具体表项值仍由图 5给出。

4.1.1 one-hot 法

实验过程与关键代码 one-hot 法从 DDT 或 LAT 中提取所有非零位置，组成传播点集 V。对每个点设置一个二进制选择变量 $\lambda_{(a,b)}$，并要求所有选择变量之和为 1，从而使每个模型解恰好选中一个传播点。

程序通过 `bits(a,b)` 计算每个传播点的八位编码，再建立八个原始变量和相应的选择变量：

```
V = [(a, b) for a in range(SIZE) for b in range(SIZE) if table[a][b] != 0]
x = model.addVars(8, vtype=gp.GRB.BINARY)
lam = model.addVars(V, vtype=gp.GRB.BINARY)
model.addConstr(lam.sum() == 1)
```

选择变量确定所选传播点后，还需将该点的坐标恢复到原始变量 x。对于第 j 位，只累加在该位取值为 1 的点所对应的选择变量：

```
for j in range(8):
    model.addConstr(x[j] == gp.quicksum(
        lam[p] for p in V if bits(*p)[j] == 1))
```

因此，每个模型包含 1 条选点等式和 8 条坐标恢复等式，共 9 条显式约束。由于每个传播点只对应一个输入输出对，选择变量与八位坐标之间是一一对应关系。

实验结果与分析 图 6给出了 DDT 和 LAT 模型的全解枚举结果。DDT 模型包含 72 个选择变量和 8 个原始变量，共 80 个二进制变量；LAT 模型包含 122 个选择变量和 8 个原始变量，共 130 个二进制变量。两个模型均包含 9 条等式，求解分别得到 72 个和 122 个解，所列输入输出对与 DDT、LAT 的非零位置逐项一致。



Figure 6: one-hot 法的 DDT、LAT 模型全解枚举结果

图 7展示一次开启求解器过程日志的 DDT 模型运行。日志显示，原始模型有 80 列（变量）和 9 行（约束）；预处理后，求解器消去了 8 个由选择变量决定的坐标变量及其坐标等式，剩余 72 个变量。该预处理仅改变求解器内部用于搜索的模型表示，报告中的原始模型规模仍按 80 个变量和 9 条约束统计。

```
(.venv) infinite@infinitedeMacBook-Air ~/Documents/crypto-analyze  main ± /Users/in
finite/Documents/crypto-analyze/.venv/bin/python /Users/infinite/Documents/crypto-analyze/l
ab4/task1/one_hot/one_hot.py

Optimize a model with 9 rows, 80 columns and 382 nonzeros (Min)
Model fingerprint: 0x14d77795
Model has 0 linear objective coefficients
Variable types: 0 continuous, 80 integer (80 binary)
Coefficient statistics:
  Matrix range     [1e+00, 1e+00]
  Objective range  [0e+00, 0e+00]
  Bounds range     [1e+00, 1e+00]
  RHS range        [1e+00, 1e+00]

Found heuristic solution: objective 0.0000000
Presolve removed 8 rows and 8 columns
Presolve time: 0.00s
Presolved: 1 rows, 72 columns, 72 nonzeros
Variable types: 0 continuous, 72 integer (72 binary)

Root relaxation: objective 0.000000e+00, 1 iterations, 0.00 seconds (0.00 work units)

    Nodes    |    Current Node    |     Objective Bounds      |     Work
 Expl Unexpl |  Obj  Depth IntInf | Incumbent    BestBd   Gap | It/Node Time

     0     0          -    0         0.00000    0.00000  0.00%     -    0s

Optimal solution found at node 0 - now completing solution pool...

    Nodes    |    Current Node    |      Pool Obj. Bounds     |     Work
             |                    |   Worst                   |
 Expl Unexpl |  Obj  Depth IntInf | Incumbent    BestBd   Gap | It/Node Time

     0     0          -    0               -    0.00000      -     -    0s
     0     0          -    0               -    0.00000      -     -    0s
     0     0          -    0               -    0.00000      -     -    0s
     0     2          -    0               -    0.00000      -     -    0s

Explored 143 nodes (3 simplex iterations) in 0.00 seconds (0.00 work units)
Thread count was 10 (of 10 available processors)

Solution count 72: 0 0 0 ... 0
No other solutions better than 0

Optimal solution found (tolerance 0.00e+00)
Best objective 0.000000000000e+00, best bound 0.000000000000e+00, gap 0.0000%

DDT one-hot: 变量数=80, 等式数=9, 解数=72
全部解（输入，输出；十六进制）：
(0,0) (1,1) (1,2) (1,5) (2,2) (2,5) (2,6) (2,9) (2,E) (3,3) (3,4) (3,7) (3,8) (3,B) (3,C) (
4,3) (4,7) (4,8) (4,C) (5,1) (5,2) (5,6) (5,9) (5,D) (6,5) (6,6) (6,A) (6,D) (6,E) (7,4) (7
,7) (7,8) (7,B) (7,F) (8,1) (8,5) (8,6) (8,9) (8,A) (9,4) (9,7) (9,8) (9,B) (A,3) (A,7) (A,
8) (A,C) (A,F) (B,2) (B,6) (B,A) (B,D) (B,E) (C,2) (C,5) (C,6) (C,9) (C,A) (C,D) (D,3) (D,4
) (D,7) (D,8) (D,B) (D,C) (E,B) (E,C) (E,F) (F,9) (F,A) (F,D) (F,E)
```

Figure 7: one-hot 法 DDT 模型的一次预处理与枚举日志

4.1.2 逐点排除法

实验过程与关键代码 逐点排除法先提取表中所有取值为 0 的位置，组成非法点列表 forbidden。模型只使用八个原始二进制变量，并为每个非法点添加一条排除不等式。由于 LAT 的正值与负值均表示可达传播，非法点统一通过表项等于 0 来提取：

```
forbidden = [(a, b) for a in range(SIZE) for b in range(SIZE)
             if table[a][b] == 0]
x = model.addVars(8, vtype=gp.GRB.BINARY)

for point in forbidden:
    q = bits(*point)
    model.addConstr(gp.quicksum(
        x[j] if q[j] == 0 else 1 - x[j]
        for j in range(8)
    ) >= 1)
```

对非法点 q，当 $q_j = 0$ 时，x_j 表示当前点在第 j 位与 q 不同；当 $q_j = 1$ 时，$1 - x_j$ 表示该位与 q 不同。将八个坐标对应的差异项相加并要求总和至少为 1，即可排除 q。因此，每条不等式只排除一个指定的非法点。

实验结果与分析 图 8给出了 DDT 和 LAT 模型的枚举结果。输入、输出各有 16 种取值，共有 $16 \times 16 = 256$ 个候选点。DDT 有 72 个合法点，因而需要排除 184 个非法点，对应 184 条不等式；LAT 有 122 个合法点，对应 134 条排除不等式。两个模型均使用 8 个二进制变量，与程序输出一致。

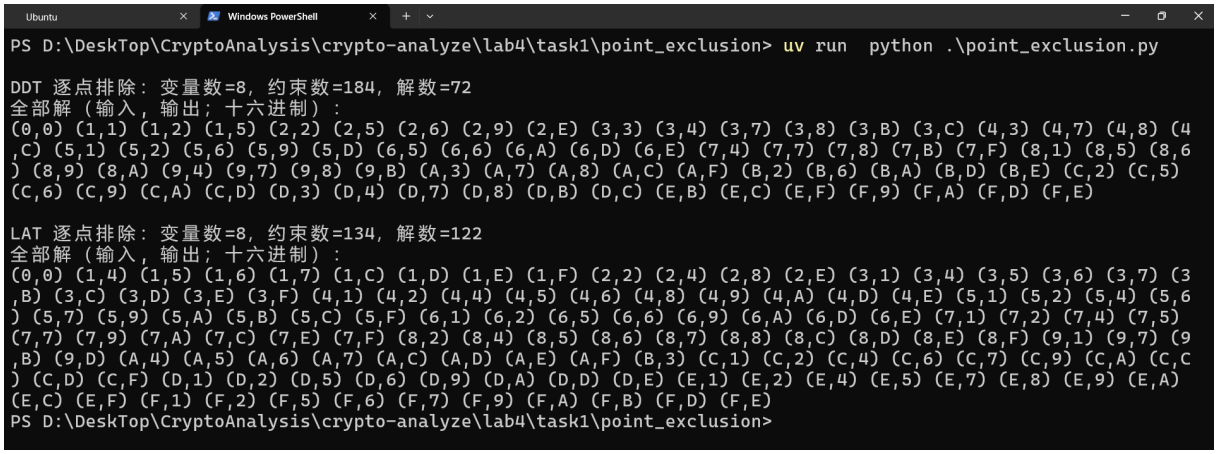


Figure 8: 逐点排除法的 DDT、LAT 模型全解枚举结果

枚举得到的 72 个差分传播点和 122 个线性传播点，与 one-hot 法及原始 DDT、LAT 中的非零位置逐点一致。该结果表明，逐点排除约束在保留八个原始变量的同时，完整刻画了两类传播点集。

4.1.3 逻辑化简法

非法点函数的构造与化简 逐点排除法为每个非法点分别建立约束。逻辑化简法则先构造八输入布尔函数，使其在非法点上取值为 1、在合法点上取值为 0，再利用 Espresso 将函数化简为若干乘积项。每个乘积项对应一个逻辑立方体，其中部分坐标固定为 0 或 1，其余坐标为通配位；一个立方体可以覆盖多个非法点。

本实验在 `logic_minimization.py` 中通过 PyEDA 的 `espresso_exprs` 接口调用 Espresso。[1]程序按公共位序建立八个布尔变量，为 DDT 或 LAT 中的每个零值位置生成一个最小项，并将所有最小项取逻辑或：

```
variables = exprvars("x", 8)
minterms = []
for a in range(SIZE):
    for b in range(SIZE):
        if table[a][b] == 0:
            point = bits(a, b)
            minterms.append(And(*(
                variable if value else ~variable
                for variable, value in zip(variables, point)
            )))

forbidden_function = Or(*minterms).to_dnf()
minimized, = espresso_exprs(forbidden_function)
```

化简后的析取范式仍表示同一个非法点函数。程序从 `minimized.to_dnf().cover` 读取各乘积项：正文字表示该位固定为 1，负文字表示固定为 0，未出现的变量表示通配位。读取文字时根据变量标识还原坐标下标，保证其顺序仍为 $(a_3, a_2, a_1, a_0, b_3, b_2, b_1, b_0)$。

立方体到线性约束的转换 对每个非法立方体 c，排除它的条件是当前点至少有一个固定坐标与 c 不同。固定为 0 的坐标贡献 x_j，固定为 1 的坐标贡献 $1 - x_j$，通配位不参与求和。程序据此为每个立方体添加一条线性不等式：

```
x = model.addVars(8, vtype=gp.GRB.BINARY)
for cube in cubes:
    model.addConstr(gp.quicksum(
        x[j] if value == 0 else 1 - x[j]
        for j, value in enumerate(cube) if value is not None
    ) >= 1)
```

因此，一条约束可以同时排除该立方体覆盖的多个非法点。Espresso 负责化简非法点函数并给出立方体覆盖；随后，程序将这些立方体转换成线性约束，由求解器枚举可行点。

实验结果与分析 图 9展示了立方体约束模型的枚举结果。本次化简为 DDT 非法点集生成 36 个立方体，为 LAT 非法点集生成 38 个立方体，因此模型分别包含 36 条和 38 条不等式。与逐点排除法的 184 条和 134 条不等式相比，逻辑化简在变量数仍为 8 的条件下减少了约束条数。

[1]PyEDA 的 Espresso 接口及两层逻辑化简说明：https://pyeda.readthedocs.io/en/latest/2llm.html。



Figure 9: 逻辑化简后由求解器枚举得到的 DDT、LAT 结果

枚举分别得到 72 个差分传播点和 122 个线性传播点，与逐点排除法及原始 DDT、LAT 中的非零位置完全一致，说明立方体覆盖及其线性化保持了原有的二进制解集。36 和 38 是本次化简得到的覆盖规模，不表示约束数已达到全局最少。

4.1.4 凸包法

利用 SageMath 构造完整凸包 凸包法包括凸包构造和约束求解两个阶段。构造脚本 `build_hull.py` 从 DDT 或 LAT 的非零位置提取传播点，在八维空间中构造凸包并生成 H 表示；求解脚本 `convex_hull.py` 读取这些约束，建立二进制模型并枚举可行点。

构造阶段将每个合法输入输出对按公共位序编码为八位向量，并在有理数域 $\mathbb{Q}$ 上进行精确计算。程序分别提取凸包的不等式和等式：

```
points = [bits(a, b) for a in range(SIZE) for b in range(SIZE)
          if table[a][b] != 0]
hull = Polyhedron(vertices=points, base_ring=QQ)

hrep = {
    "inequalities": sorted(integer_row(row)
                           for row in hull.inequalities_list()),
    "equations": sorted(integer_row(row)
                        for row in hull.equations_list()),
}
```

SageMath 输出的每行系数按 $[c_0, c_1, \ldots, c_8]$ 排列，其中 c_0 为常数项，对应的不等式为 $c_0 + \sum_{j=0}^{7} c_{j+1} x_j \geq 0$；等式采用相同的系数顺序。[2]为将系数保存为整数，`integer_row()` 先乘以分母的最小公倍数，再除以整数系数的最大公因数。缩放因子为正数，因此不等式方向保持不变。

完整 H 表示保存在 `hull_constraints.json` 中。JSON 文件以 `DDT` 和 `LAT` 为顶层键，每组包含 `inequalities` 和 `equations` 两个系数数组。文件位于 Windows 工作目录，WSL 通过`/mnt/d/`路径访问并生成数据，Windows 端随后读取同一文件。

贪心筛选与模型求解 完整 H 表示准确描述凸包，但本任务关注其二进制可行点，因此在保留全部等式的前提下，进一步筛选能够排除非法二进制点的不等式。程序先将不满足等式的非法点移出待覆盖集合 `remaining`，再由 `pool` 记录每条候选不等式所能排除的点。每轮选择覆盖当前最多剩余非法点的不等式，并从 `remaining` 中移除已排除点，直至待覆盖集合为空。核心循环如下：

```
while remaining:
    row = max(pool, key=lambda item: len(pool[item] & remaining),
              default=None)
    if row is None or not (pool[row] & remaining):
        raise RuntimeError("Incomplete illegal-point coverage")
    chosen.append(row)
    remaining.difference_update(pool.pop(row))
```

函数 `linear_form(row,point)` 计算不等式在指定点上的取值；若结果小于 0，该点违反此不等式并被排除。候选行按系数顺序排列，使并列时的选择固定。每条保留约束均来自完整 H 表示，因而合法点仍满足所有保留约束；循环结束时全部非法点也均已排除。

最终模型使用八个二进制变量，将全部等式以 $h(x) = 0$、筛选后的不等式以 $h(x) \geq 0$ 加入模型，并按照统一的解池设置枚举全部可行点。

[2]SageMath 多面体构造与 H 表示说明：https://doc.sagemath.org/html/en/reference/discrete_geometry/sage/geometry/polyhedron/constructor.html。

实验结果与分析 图 10展示了 SageMath 构造完整凸包并保存 H 表示的输出。DDT 凸包包含 145 条不等式，LAT 凸包包含 134 条不等式，两者的等式数均为 0，与两组传播点集的仿射维数均为 8 相符。对完整 H 表示进行二进制点核对，分别得到 72 个和 122 个可行点，与原始表中的非零位置一致。

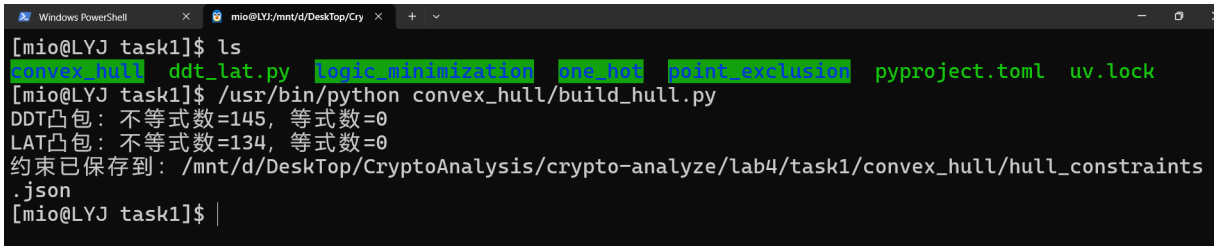


Figure 10: SageMath 构造 DDT、LAT 传播点集的凸包并保存 H 表示

图 11给出约束筛选后的枚举结果。DDT 模型从 145 条不等式中保留 32 条，LAT 模型从 134 条中保留 28 条；由于两组模型均无等式，最终约束数分别为 32 和 28，变量数均为 8。枚举得到 72 个差分传播点和 122 个线性传播点，与其余三种方法及 DDT、LAT 的非零位置逐点相同。



Figure 11: 凸包约束经贪心筛选后的全解枚举结果

此外，对全部 256 个二进制点逐一代入约束后，完整 H 表示及筛选后的约束均未遗漏合法点，也未保留非法点。这验证了约束读取、整数化和贪心筛选对二进制解集的保持性。筛选后的连续可行域可能大于完整凸包；贪心过程减少了本次模型的约束数，但不保证约束数达到全局最少。

4.1.5 四种方法的结果对比

表 1汇总了四种方法的变量数、显式线性约束数和全解枚举数。变量数包含模型中的二进制变量；约束数统计显式添加的线性约束，凸包法统计经贪心筛选后实际加入模型的不等式和等式。

Table 1: 四种方法的模型规模与全解枚举结果

方法	DDT 模型			LAT 模型		
	变量数	约束数	解数	变量数	约束数	解数
one-hot 法	80	9	72	130	9	122
逐点排除法	8	184	72	8	134	122
逻辑化简法	8	36	72	8	38	122
凸包法	8	32	72	8	28	122

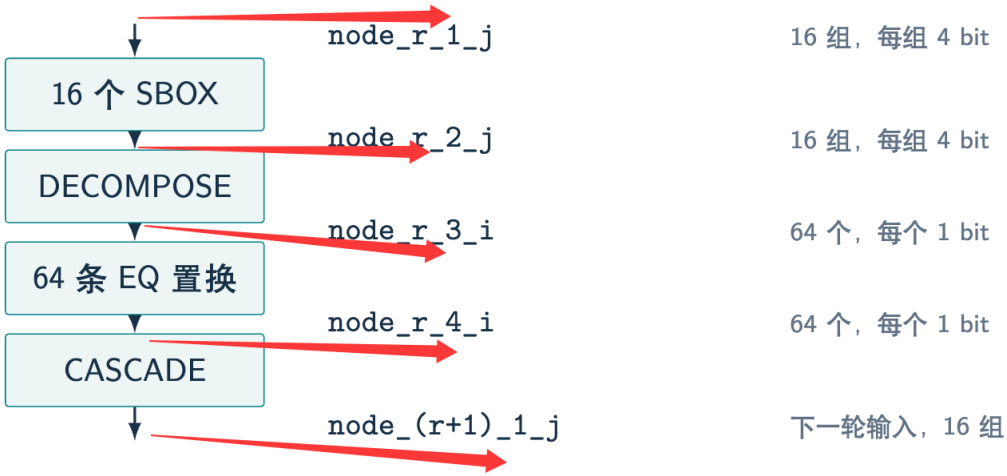
四种方法均枚举得到 DDT 的 72 个和 LAT 的 122 个非零位置，与直接计算结果一致。one-hot 法使用额外的选择变量表达点集，其余三种方法只使用八个传播变量；本次凸包法经过贪心筛选后的约束数最少。Espresso 化简和贪心筛选均为本次实验采用的约简过程，得到的约束数不代表全局最小值；约束数量也不能单独用来推断求解速度。

4.2 任务二：Cipherfour 算法六轮最优差分与线性路线搜索

4.2.1 实验步骤

（1）算法建模与平台配置 本任务利用密码分析学课程实验平台，对 Cipherfour 算法分别进行六轮最优差分路线搜索与最优线性路线搜索。实验首先需要根据算法的轮函数结构编写平台配置文件，并通过平台提供的组件描述各轮之间的传播关系。因此，在正式配置算法之前，先对平台的变量命名规则及组件功能进行分析。

根据课程提供的使用指南，平台使用 `node_r_l_j` 的形式命名状态变量，其中 r 表示轮编号，l 表示层编号，j 表示该层中的变量位置。以课件中的 PRESENT 算法为例，每轮可以划分为 S 盒输入、S 盒输出、比特拆分及置换输出四个状态层，各层之间通过相应组件建立连接，具体结构如下图所示。



r：轮编号；*j*：组编号（0–15）；*i*：比特位置（0–63）。
差分模式中，变量记录差分；线性模式中，变量记录掩码。

具体而言，第一层记录进入 S 盒之前的状态，第二层记录经过 S 盒替换后的输出状态。随后，通过 DECOMPOSE 组件将 S 盒输出拆分为单比特变量，形成第三层状态，以便对各个比特进行独立的置换操作。第三层与第四层之间通过 EQ 组件建立比特位置对应关系，完成置换后，再由 CASCADE 组件将单比特状态重新组合为下一轮 S 盒的输入。

在差分分析中，同一密钥下的轮密钥异或不会改变差分值，即

$$(x \oplus k) \oplus (x' \oplus k) = x \oplus x'.$$

对于线性分析，固定轮密钥异或仅影响线性相关度的符号，不改变其绝对值。因此，在本次差分概率和线性相关度绝对值的路线搜索中，可以省略轮密钥异或组件，重点描述 S 盒替换及轮间比特置换。

在平台配置文件中，首先通过 PARAMETERS 定义算法使用的 S 盒参数，再利用 SBOX、DECOMPOSE、EQ 和 CASCADE 等组件描述各轮操作。课程提供的 PRESENT 配置示例已经展示了上述组件的组织方式，因此本实验以该示例为基础，根据 Cipherfour 的实际结构调整状态位宽、S 盒数量以及轮间连接关系。

```
SETTINGS:
START_NODES = [<node_0_1_i,4,i,0,4,1>]
END_NODES =  <node_5_2_i 4,i,0,4,1>]

START_POSITION = 0
END_POSITION = 0
START_VALUE = 0x8
END_VALUE = 0x8

DIFF_UP_BOUND = 128
LINEAR_UP_BOUND = 64
ENDSETTINGS

PARAMETERS:
s = [0x9,0x8,0x0,0x1,0xE,0xB,0xC,0xD,0x3,0x2,0x5,0x7,0xF,0xA,0x6,0x4]
ENDPARAMETERS
```

与 PRESENT 算法的 64 bit 分组和 16 个并行 S 盒不同，Cipherfour 的分组长度为 16 bit，每轮由 4 个并行的 4 bit S 盒组成。因此，配置文件中需要将每轮 S 盒的循环次数调整为 4，并使用实验题目指定的 S 盒参数。Cipherfour 的轮函数结构如下图所示。

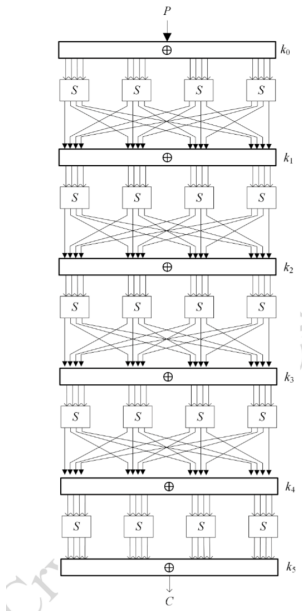


图 A.4: 5轮CipherFour算法加密示意图

此外，本次使用的 Cipherfour 结构在最后一轮 S 盒替换后不再执行 P 置换，因此六轮模型中的前五轮需要描述 S 盒替换与轮间置换，而最后一轮只需保留 S 盒替换操作。由于平台的轮编号从 0 开始，第六轮对应 $r = 5$，其 S 盒输出位于第二层，因此模型的终止状态设置为

`node_5_2_i`.

对于搜索参数，`DIFF_UP_BOUND` 和 `LINEAR_UP_BOUND` 分别用于设置差分分析与线性分析的初始搜索权重上界。本实验分别采用 128 和 64 作为初始上界，并通过平台的任

务类型选择相应的搜索目标。两类分析使用相同的算法结构，但平台会根据所选任务采用不同的 S 盒传播约束及优化指标。

```
SETTINGS:
START_NODES = [<node_0_1_i,4,i,0,4,1>]
END_NODES = [<node_5_2_i,4,i,0,4,1>]

START_POSITION = 0
END_POSITION = 0
START_VALUE = 0x8
END_VALUE = 0x8

DIFF_UP_BOUND = 128
LINEAR_UP_BOUND = 64
ENDSETTINGS

PARAMETERS:
```

配置文件还包含 START_POSITION、END_POSITION、START_VALUE 和 END_VALUE 等边界参数。本次实验沿用课件中的示例设置，最终通过平台输出的路线及搜索结果进行核验。需要注意，这些参数是否实际参与搜索约束取决于平台所采用的具体任务模式，不能仅根据其配置值判断最终路线的起止状态是否受到限制。

（2）提交任务与调试 完成算法配置后，将配置文件保存为 UTF-8 编码的文本文件，并上传至密码分析学课程实验平台。首先选择“概率最优差分路线”任务，对 Cipherfour 进行六轮差分路线搜索；随后保留相同的算法配置，将任务类型切换为“相关度最优线性路线”，完成对应的线性分析。

在首次提交差分任务时，平台显示任务失败，但未提供具体错误日志，因此无法直接确定问题来源。随后重新检查配置文件中的组件语法、循环范围、变量编号及轮间连接关系，并对部分配置进行调整。修改后，差分搜索成功运行；在此基础上继续提交线性搜索任务，也顺利获得了对应的分析结果。

本次实验最终完成了两项搜索任务，平台申请记录如图 12 所示。

（3）搜索结果提取与理论验证 平台运行结束后，分别下载差分分析和线性分析对应的 .rpt 与 .cvc 文件。其中，.rpt 文件主要记录搜索迭代过程、求解状态、各轮输入输出差分或线性掩码，以及相应的概率或相关度指标；.cvc 文件则保存平台生成的约束求解模型，可用于进一步检查传播约束及权重表示方式。

根据差分搜索报告，平台首先在配置文件给定的初始权重上界 128 下寻找可行路线。在首次成功搜索中，平台找到了一条概率为 2^{-26} 的六轮差分路线，其对应的搜索日志如下图所示。

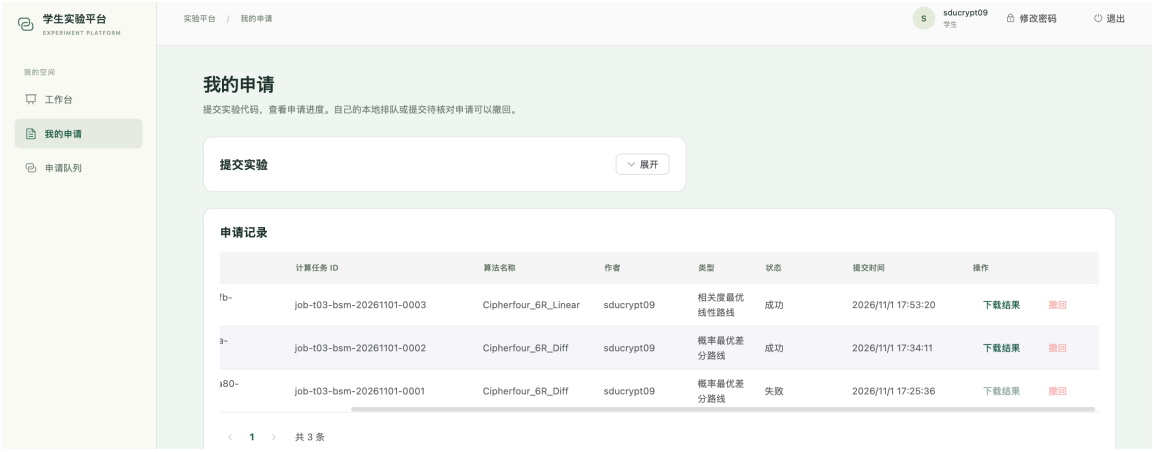
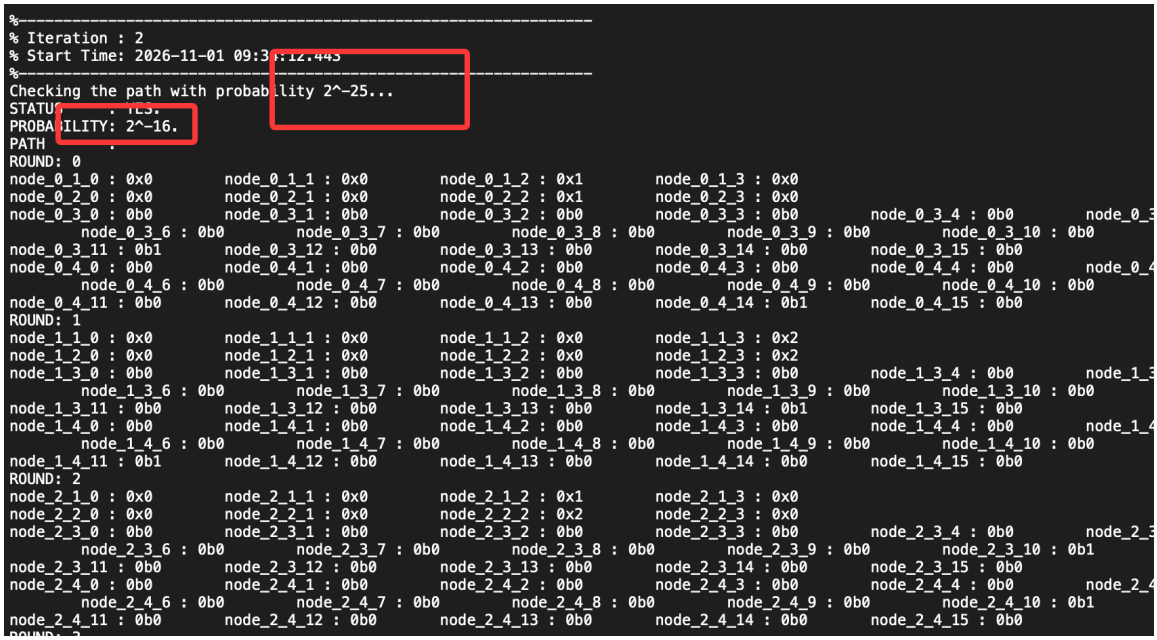


Figure 12: Cipherfour 差分与线性路线搜索任务的提交及运行结果

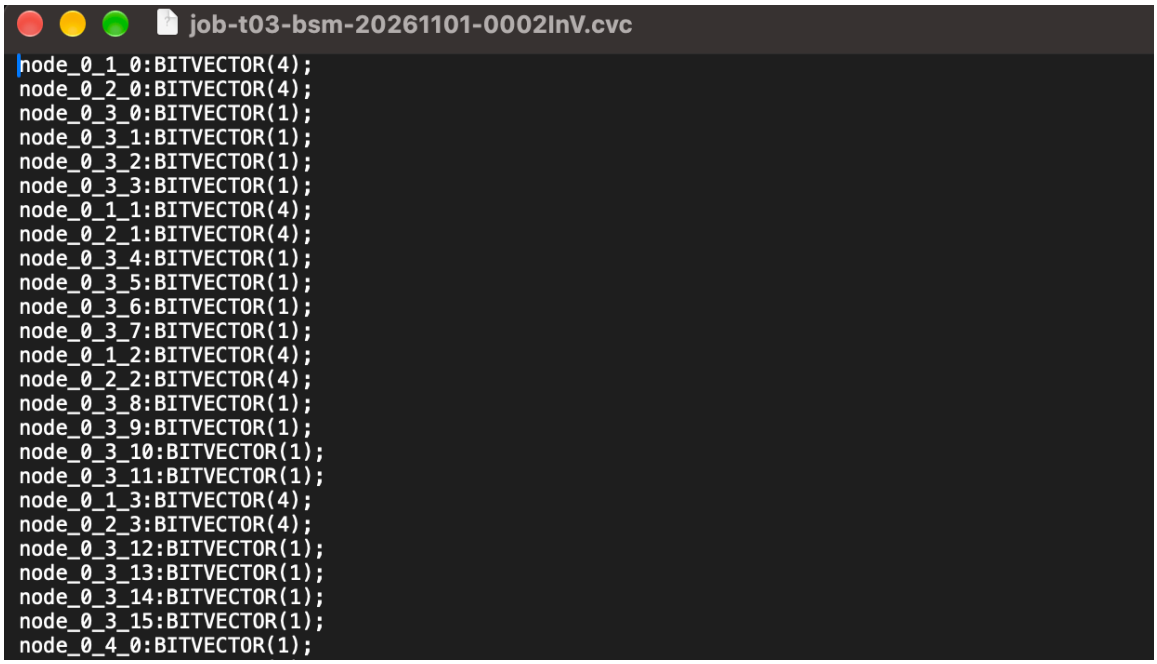
```
%------------------------------------------------------------
% Cipher Name: Cipherfour_6R_Diff
% Attack Mode: _DIFF_PATH
% Start  Time: 2026-11-01 09:34:11.963
%------------------------------------------------------------

%------------------------------------------------------------
% Iteration : 1
% Start Time: 2026-11-01 09:34:11.978
%------------------------------------------------------------
Checking the path with probability 2^-128...
STATUS     : YES.
PROBABILITY: 2^-26.
PATH       :
ROUND: 0
node_0_1_0 : 0x3        node_0_1_1 : 0x0        node_0_1_2 : 0x2        node_0_1_3 : 0x0
node_0_2_0 : 0x3        node_0_2_1 : 0x0        node_0_2_2 : 0x2        node_0_2_3 : 0x0
node_0_3_0 : 0b0        node_0_3_1 : 0b0        node_0_3_2 : 0b1        node_0_3_3 : 0b1        node_0_3_4 : 0b0        noc
      node_0_3_6 : 0b0        node_0_3_7 : 0b0        node_0_3_8 : 0b0        node_0_3_9 : 0b0        node_0_3_10 : 0b1
node_0_3_11 : 0b0       node_0_3_12 : 0b0       node_0_3_13 : 0b0       node_0_3_14 : 0b0       node_0_3_15 : 0b0
node_0_4_0 : 0b0        node_0_4_1 : 0b0        node_0_4_2 : 0b0        node_0_4_3 : 0b0        node_0_4_4 : 0b0        noc
      node_0_4_6 : 0b0        node_0_4_7 : 0b0        node_0_4_8 : 0b1        node_0_4_9 : 0b0        node_0_4_10 : 0b1
node_0_4_11 : 0b0       node_0_4_12 : 0b1       node_0_4_13 : 0b0       node_0_4_14 : 0b0       node_0_4_15 : 0b0
ROUND: 1
node_1_1_0 : 0x0        node_1_1_1 : 0x0        node_1_1_2 : 0xA        node_1_1_3 : 0x8
node_1_2_0 : 0x0        node_1_2_1 : 0x0        node_1_2_2 : 0x8        node_1_2_3 : 0xA
node_1_3_0 : 0b0        node_1_3_1 : 0b0        node_1_3_2 : 0b0        node_1_3_3 : 0b0        node_1_3_4 : 0b0        noc
      node_1_3_6 : 0b0        node_1_3_7 : 0b0        node_1_3_8 : 0b1        node_1_3_9 : 0b0        node_1_3_10 : 0b0
node_1_3_11 : 0b0       node_1_3_12 : 0b1       node_1_3_13 : 0b0       node_1_3_14 : 0b1       node_1_3_15 : 0b0
node_1_4_0 : 0b0        node_1_4_1 : 0b0        node_1_4_2 : 0b1        node_1_4_3 : 0b1        node_1_4_4 : 0b0        noc
      node_1_4_6 : 0b0        node_1_4_7 : 0b0        node_1_4_8 : 0b0        node_1_4_9 : 0b0        node_1_4_10 : 0b0
node_1_4_11 : 0b1       node_1_4_12 : 0b0       node_1_4_13 : 0b0       node_1_4_14 : 0b0       node_1_4_15 : 0b0
ROUND: 2
node_2_1_0 : 0x3        node_2_1_1 : 0x0        node_2_1_2 : 0x1        node_2_1_3 : 0x0
```

在获得可行路线后，平台进一步收紧搜索条件，将下一次搜索的概率阈值调整为 2^{-25}，继续寻找具有更高传播概率的路线。通过不断迭代，平台逐步提高候选路线的概率要求，直至无法找到满足更严格约束的路线，从而确定当前搜索模型下的最优差分权重。下一次迭代的运行情况如下图所示。



除搜索报告外，平台还生成了对应的 .cvc 约束文件。从文件内容可以观察到，平台将各轮状态变量及传播关系转化为位向量约束，并采用 STP/CVC 风格的语法组织相关表达式。其基本思想与任务一中使用 Gurobi 建立的约束模型具有相似性，即通过数学约束限定合法传播，再根据权重条件判断是否存在满足要求的路线。相关约束文件内容如下图所示。



需要说明的是，虽然 .cvc 文件采用了与 STP 兼容的约束表示方式，但仅根据文件语法尚不能完全确定平台内部的具体求解器实现。因此，本实验主要将其用于分析平台生成的传播约束和权重编码方式，而不直接据此判断平台的完整求解流程。

在理论验证阶段，首先从 .rpt 文件中提取各轮输入输出差分及线性掩码，再分别利用任务一计算得到的 DDT 和 LAT 检查 S 盒局部传播的合法性与对应概率或相关度。同

时，根据轮间比特置换关系检查相邻两轮状态是否正确连接，最后计算整条路线的概率或相关度，并将理论结果与平台输出进行比较，以验证搜索路线的准确性。

4.2.2 实验结果及分析

（1）六轮最优差分路线搜索 差分分析任务的名称为 `Cipherfour_6R_Diff`。

平台采用逐步收紧概率下界的方式进行搜索。在第 9 次迭代中，平台找到概率为 2^{-6} 的可行差分路线；第 10 次迭代进一步尝试搜索概率不低于 2^{-5} 的路线，但求解结果为不可行。

对应的关键运行结果如下：

```
Iteration : 9
Checking the path with probability 2^-6...
STATUS      : YES.
PROBABILITY: 2^-6.

Iteration : 10
Checking the path with probability 2^-5...
STATUS      : No.
```

由此可知，在当前六轮差分搜索模型下，最优差分路线的权重为

$$W_D = 6$$

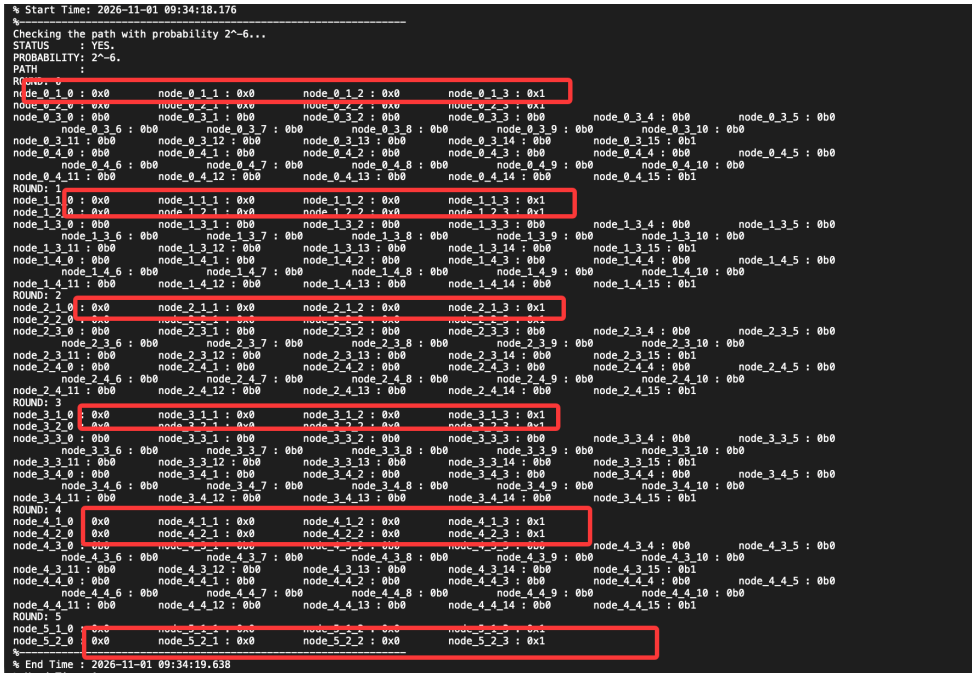
对应的差分路线概率为

$$P_{\text{trail}} = 2^{-6} = \frac{1}{64}$$

从第 9 次迭代的输出中提取六轮差分路线，如表 2 所示。

Table 2: Cipherfour 六轮最优差分路线

轮数	输入差分	输出差分	轮传播概率
1	0x0001	0x0001	2^{-1}
2	0x0001	0x0001	2^{-1}
3	0x0001	0x0001	2^{-1}
4	0x0001	0x0001	2^{-1}
5	0x0001	0x0001	2^{-1}
6	0x0001	0x0001	2^{-1}



可以发现，该路线的六轮输入输出差分均为 0x0001。每轮只有最低四比特对应的 S 盒处于活跃状态，其余三个 S 盒的输入输出差分均为 0。

根据任务一计算得到的 DDT，有

$$\mathrm{DDT}[1][1] = 8, \qquad \mathrm{DDT}[0][0] = 16.$$

因此，活跃 S 盒的差分传播概率为

$$\Pr(1 \to 1) = \frac{\mathrm{DDT}[1][1]}{16} = \frac{8}{16} = \frac{1}{2}.$$

对于其他三个非活跃 S 盒，有

$$\Pr(0 \to 0) = 1.$$

因此，每轮四个 S 盒的联合传播概率为

$$P_r = 1 \times 1 \times 1 \times \frac{1}{2} = \frac{1}{2}.$$

同时，由于该路线的非零差分位位于整个状态的最低位，在所采用的置换中有

$$\pi(15) = 15,$$

因此该差分位经过置换后仍保持在原位置，可以在下一轮继续进行相同的传播。

综合六轮的传播概率，根据差分路线的概率乘积模型，有

$$P_{\text{trail}} = \prod_{r=1}^{6} P_r = \left(\frac{1}{2}\right)^6 = 2^{-6}$$

因此，理论计算结果与平台报告的差分路线概率完全一致。

结合第 10 次迭代中更高概率阈值下无可行路线的结果，可以确认平台在当前差分模型下找到了最优六轮差分路线。

（2）六轮最优线性路线搜索 线性分析任务的名称为 `Cipherfour_6R_Linear`。

与差分分析类似，平台逐步收紧相关度对应的权重上界。

在第 9 次迭代中，平台找到内部权重为 3 的可行线性路线，并报告相关度指标为 2^{-3}；第 10 次迭代继续搜索权重不超过 2 的路线，结果为不可行。

关键运行结果如下：

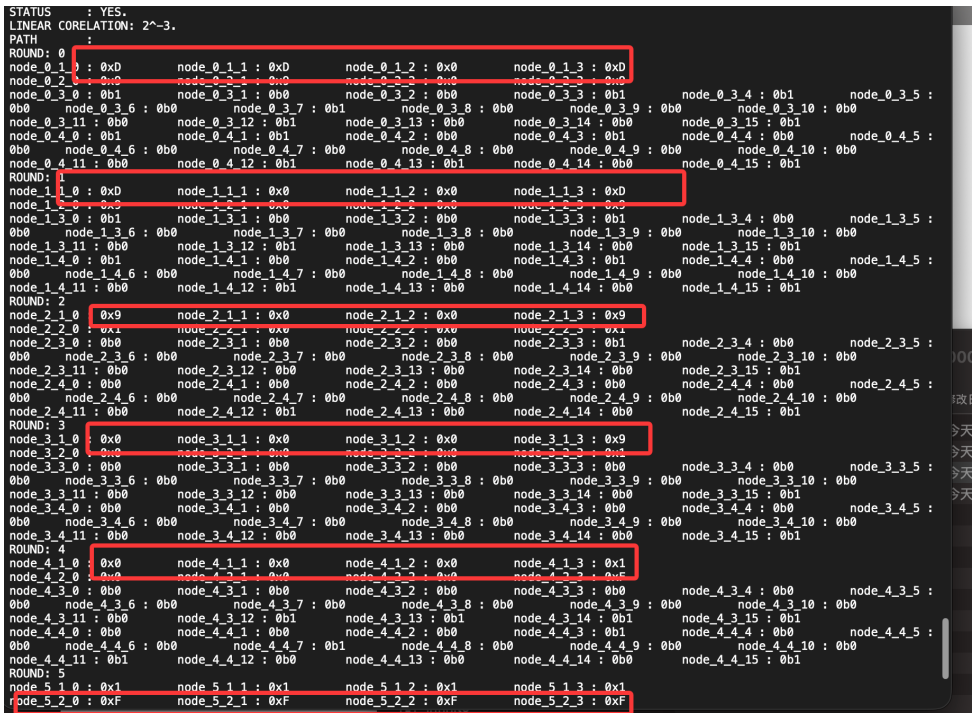
```
Iteration : 9
Checking the path with correlation of 2^-3...
STATUS      : YES.
LINEAR CORELATION: 2^-3.

Iteration : 10
Checking the path with correlation of 2^-2...
STATUS      : No.
```

因此，平台在其内部整数权重模型下得到的最优值为

$$\widetilde{W}_L = 3$$

第 9 次迭代输出的六轮线性路线如表 3 所示。



为了验证上述路线，首先根据任务一的 LAT 计算各个 S 盒输入输出掩码对的相关度。对于输入掩码 α 和输出掩码 β，定义 S 盒的 Walsh 相关和为

$$W(\alpha,\beta) = \sum_{x\in\mathbb{F}_2^4} (-1)^{\alpha\cdot x \oplus \beta\cdot S(x)}.$$

其对应的相关度为

$$C(\alpha,\beta) = \frac{W(\alpha,\beta)}{16}.$$

Table 3: Cipherfour 平台搜索得到的六轮线性路线

轮数	输入掩码	输出掩码	精确轮相关度绝对值
1	0xDD0D	0x9909	$(3/4)^3$
2	0xD00D	0x9009	$(3/4)^2$
3	0x9009	0x1001	$(1/2)^2$
4	0x0009	0x0001	$1/2$
5	0x0001	0x000F	$3/4$
6	0x1111	0xFFFF	$(3/4)^4$

若 LAT 使用偏差计数 $\mathrm{LAT}[\alpha][\beta] = N_0 - 8$ 的定义，则有

$$W(\alpha, \beta) = 2\,\mathrm{LAT}[\alpha][\beta].$$

观察平台给出的线性路线，可以发现其中只涉及以下四种 S 盒局部传播：

Table 4: 线性路线所涉及的 S 盒局部相关度

输入掩码	输出掩码	Walsh 值	相关度
0x0	0x0	16	1
0xD	0x9	12	3/4
0x9	0x1	8	1/2
0x1	0xF	12	3/4

由表 4 可知，上述局部传播的相关度均不为零，因此均属于合法的线性掩码传播。

进一步检查相邻两轮的掩码关系，发现各轮 S 盒输出掩码经过比特置换后，均与下一轮 S 盒的输入掩码一致，说明该路线满足算法的轮间连接约束。

按照线性路线相关度的乘积公式，

$$C_{\text{trail}} = \prod_{r=1}^{6} \prod_{j=0}^{3} C_{r,j},$$

其中 $C_{r,j}$ 表示第 r 轮第 j 个 S 盒的局部相关度。

对该路线中的局部传播进行统计，共有 10 个相关度为 $3/4$ 的传播、3 个相关度为 $1/2$ 的传播，其余 11 个位置的相关度均为 1。

因此，

$$\begin{aligned} |C_{\text{trail}}| &= \left(\frac{3}{4}\right)^{10} \left(\frac{1}{2}\right)^{3} \\ &= \frac{59049}{8388608} \\ &\approx 0.00703919 \end{aligned}$$

对应的精确线性权重为

$$\begin{aligned} W_L &= -\log_2 |C_{\text{trail}}| \\ &\approx 7.1504 \end{aligned}$$

需要注意的是，精确计算得到的线性权重与平台报告的整数权重 3 并不相同。

为分析这一差异，进一步检查平台生成的 .cvc 文件中的 S 盒权重编码，发现平台对非零相关度使用的权重符合

$$\widetilde{w}(\alpha,\beta) = \lfloor -\log_2 |C(\alpha,\beta)| \rfloor .$$

例如，对于相关度为 $3/4$ 的局部传播，

$$-\log_2 \frac{3}{4} \approx 0.415,$$

经过向下取整后，平台将其编码为

$$\widetilde{w} = 0.$$

而对于相关度为 $1/2$ 的局部传播，则有

$$\widetilde{w} = -\log_2 \frac{1}{2} = 1.$$

因此，该路线在平台内部的总权重为

$$\begin{aligned}\widetilde{W}_L &= 10 \times 0 + 3 \times 1 + 11 \times 0 \\ &= 3\end{aligned}$$

这一结果与平台报告完全一致。

由此说明，平台报告的 2^{-3} 是其整数化权重模型对应的搜索指标，而根据原始 LAT 计算得到的该路线精确相关度绝对值约为 0.00703919。

因此，平台成功找到了整数化权重模型下的最优线性路线，但该结果不能直接作为精确相关度全局最优性的证明。

（3）实验结果总结 本次实验利用课程自动化分析平台，完成了 Cipherfour 算法的六轮差分路线和六轮线性路线搜索，并根据任务一计算得到的 DDT 和 LAT 对搜索结果进行了独立验证

两项搜索结果汇总如表 5 所示。

Table 5: Cipherfour 六轮差分与线性搜索结果汇总

分析类型	平台最优权重	平台报告指标	精确验证结果
差分分析	6	2^{-6}	2^{-6}
线性分析	3	2^{-3}	0.00703919

5 实验总结

本次实验围绕 Cipherfour 算法的差分传播与线性传播分析展开，主要完成了 S 盒差分分布表（DDT）和线性分布表（LAT）的计算、四种传播约束模型的构造与验证，以及六轮差分路线和线性路线的自动化搜索。通过将 S 盒传播关系转化为有限二进制点集，并进一步使用线性约束对其进行刻画，实现了从密码学传播性质到约束求解模型的转换。

在任务一中，分别使用 One-hot 法、逐点排除法、Espresso 逻辑化简法及凸包法构造 S 盒传播模型，并通过 Gurobi 枚举全部二进制可行解。实验结果表明，四种方法均能够准确描述 DDT 的 72 个合法传播点和 LAT 的 122 个合法传播点，但不同方法在辅助变量数量与约束规模方面存在明显差异。其中，One-hot 法的建模形式较为直接，逐点排除法的正确性容易验证，而逻辑化简和凸包约简能够在不改变二进制可行解集合的前提下有效减少约束数量。这说明，在保证模型正确性的基础上，进一步优化约束表示是提高自动化分析效率的重要环节。

在任务二中，利用课程实验平台完成了 Cipherfour 的六轮最优差分路线与线性路线搜索，并结合任务一计算得到的 DDT 和 LAT 对结果进行独立核验。差分分析得到的最优路线概率为 2^{-6}，与理论计算完全一致。线性分析则发现，平台报告的整数化权重与根据 LAT 计算得到的精确相关度存在差异。通过进一步检查平台生成的约束文件，分析了这一差异与局部传播权重表示方式之间的关系，也认识到自动化搜索结果的最优性需要在具体模型及目标函数的定义下进行判断。

总体而言，本次实验加深了对差分分析、线性分析及 MILP 建模方法之间联系的理解，也使我更加熟悉了从传播点集构造、线性约束生成到求解器验证的完整流程。特别是在自动化平台的配置与结果分析过程中，进一步认识到准确描述算法结构、统一比特位序以及独立验证求解结果的重要性。